

Day 87: MongoDB Basics

1. Day Number

Day 87

2. Topic Name

Document Databases and MongoDB

Today you will learn a different way to store data.

Yesterday, you worked with **MySQL**, where data is usually stored in relational tables containing rows and columns.

Today, you will learn about **MongoDB**, which stores data as **documents** that look similar to Python dictionaries.

3. Connection

You already know that MySQL can store structured data like this:

```
products
-----
id | name      | price | stock
1  | Keyboard  | 1200  | 10
2  | Mouse     | 500   | 25
```

MongoDB uses a different structure.

Instead of thinking mainly in rows and columns, you can think in dictionary-like documents:

```
{
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

So the connection is:

```
MySQL
→ tables
→ rows
→ columns

MongoDB
→ collections
→ documents
→ fields
```

4. Important Topics

NoSQL Idea

MongoDB is commonly described as a **NoSQL database**.

At a beginner level, this means it does not primarily organize data using the traditional relational table structure used by databases such as MySQL.

This does **not** mean that the data has no structure.

MongoDB documents still contain organized fields and values.

For example:

```
{
  "name": "Monitor",
  "price": 9000,
  "stock": 5
}
```

Database

A MongoDB **database** can contain multiple collections.

For example:

```
shop_db

  products
  customers
  orders
```

Here, `shop_db` is the database.

Collection

A **collection** stores related documents.

For example:

```
products collection
```

could contain:

```
{"name": "Keyboard", "price": 1200}
{"name": "Mouse", "price": 500}
{"name": "Monitor", "price": 9000}
```

You can loosely think of a collection as being similar to a SQL table, although they are not exactly the same.

Document

A **document** is one stored record.

Example:

```
{
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

This is conceptually similar to one row in a relational database.

_id

MongoDB documents normally have a special field called:

```
_id
```

It uniquely identifies the document.

Conceptually:

```
{
  "_id": some_unique_value,
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

If you do not provide **_id** when inserting a document, MongoDB normally creates one automatically.

At this stage, simply remember:

| **_id** uniquely identifies a MongoDB document.

Insert

To add a document, you perform an **insert**.

Conceptually in Python:

```
product = {
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

```
}  
  
products.insert_one(product)
```

Think:

```
insert_one()  
→ store one document
```

Find

To retrieve documents, MongoDB provides operations such as:

```
find_one()
```

For example:

```
products.find_one({"name": "Keyboard"})
```

Conceptually this means:

| Find one document whose `name` field is `"Keyboard"`.

5. Foundational Notes

MongoDB stores data using documents that are similar to JSON-style objects.

A document consists of:

```
field → value
```

For example:

```
{  
  "name": "Laptop",  
  "price": 55000,  
  "stock": 4  
}
```

Here:

```
name → Laptop  
price → 55000  
stock → 4
```

Unlike the SQL syntax you learned earlier, MongoDB operations in Python often involve Python dictionaries.

That can make the basic idea feel familiar because you already know dictionaries.

6. MongoDB Document vs Python Dictionary

A Python dictionary might look like:

```
product = {  
    "name": "Mouse",  
    "price": 500,  
    "stock": 20  
}
```

A MongoDB document can look conceptually almost the same:

```
{  
  "name": "Mouse",  
  "price": 500,  
  "stock": 20  
}
```

The similarity is important.

Both contain:

```
keys / fields  
+  
values
```

For example:

```
"name" → "Mouse"  
"price" → 500  
"stock" → 20
```

One important difference is that a Python dictionary exists in your Python program's memory, while a MongoDB document can be stored permanently in the database.

So:

```
Python dictionary  
→ data inside your running Python program  
  
MongoDB document  
→ data stored in MongoDB
```

7. MongoDB vs Relational Table

Suppose we have product data.

Relational database idea

In MySQL:

```
products
-----
id | name      | price | stock
1  | Keyboard  | 1200  | 10
2  | Mouse     | 500   | 25
```

Each product is a **row**.

The table has defined **columns**.

MongoDB idea

MongoDB might store:

```
{
  "_id": 1,
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

and:

```
{
  "_id": 2,
  "name": "Mouse",
  "price": 500,
  "stock": 25
}
```

Each product is a **document** inside a **collection**.

A simple comparison is:

Relational database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field

Relational database	MongoDB
Primary ID	<code>_id</code>

This comparison is useful for learning, even though MongoDB and relational databases work differently in many deeper ways.

8. Easy Example

Imagine a MongoDB database called:

```
shop_db
```

Inside it is a collection:

```
products
```

You want to store:

```
product = {
    "name": "Keyboard",
    "price": 1200,
    "stock": 8
}
```

Conceptually, Python could insert it using:

```
products.insert_one(product)
```

Then you could search for it using:

```
products.find_one({"name": "Keyboard"})
```

The result might conceptually look like:

```
{
    "_id": ...,
    "name": "Keyboard",
    "price": 1200,
    "stock": 8
}
```

Notice that MongoDB may have added `_id`.

9. Problem Statement

Create a very small MongoDB exercise using product data.

Your product should contain:

```
name
price
stock
```

For example:

```
{
  "name": "Monitor",
  "price": 9000,
  "stock": 6
}
```

Your task is to conceptually:

1. connect to MongoDB
2. choose a database
3. choose a `products` collection
4. create one product dictionary
5. insert the product document
6. search for the product by name
7. print the returned document

Do not add advanced MongoDB features.

10. Concepts Used

For this exercise, you will use:

- MongoDB
 - NoSQL idea
 - database
 - collection
 - document
 - field
 - value
 - `_id`
 - Python dictionary
 - insert
 - `insert_one()` concept
 - find
 - `find_one()` concept
-

11. Thought Process

Start by asking:

Where should the data go?

You need a MongoDB database, such as:

```
shop_db
```

What type of information are you storing?

Products.

Therefore you can use a collection named:

```
products
```

What does one product need?

The problem says:

```
name  
price  
stock
```

So construct a dictionary:

```
{  
    "name": ...,  
    "price": ...,  
    "stock": ...  
}
```

What operation stores it?

Use the idea of:

```
insert_one()
```

How do you retrieve it?

Search using a field that identifies the product.

For example:

```
name = "Monitor"
```

Conceptually:

```
find_one({"name": "Monitor"})
```

What comes back?

Either:

```
a matching document
```

or:

```
no document
```

Your program should be ready for both cases.

12. Beginner-Friendly Pseudocode

```
START

connect to MongoDB

select shop database

select products collection

create product dictionary
    name = "Monitor"
    price = 9000
    stock = 6

insert product into products collection

search products collection
    where name is "Monitor"

IF product was found:
    print product
ELSE:
    print "Product not found"

END
```

A useful sequence to remember is:

```
Connect
  ↓
Database
  ↓
```

```
Collection
  ↓
Document
  ↓
Insert
  ↓
Find
```

13. Easy Edge Cases

Document Not Found

Suppose you search for:

```
{"name": "Printer"}
```

but there is no printer document.

Conceptually, `find_one()` will not give you a product document.

Your program should handle that situation:

```
IF result exists:
    print result
ELSE:
    print "Product not found"
```

An empty search result is not necessarily an error.

It may simply mean the document does not exist.

Missing Field

Suppose you have this document:

```
{
    "name": "Mouse",
    "price": 500
}
```

Notice:

```
stock
```

is missing.

Trying to directly access a missing dictionary-like field can cause problems depending on how your Python code accesses it.

Instead of assuming every document has every field, you can eventually use safer dictionary techniques such as:

```
document.get("stock")
```

For now, the important lesson is:

| Do not automatically assume that every document contains every possible field.

14. Expected Document / Result

Before insertion, your Python dictionary might be:

```
{
    "name": "Monitor",
    "price": 9000,
    "stock": 6
}
```

After being stored in MongoDB, the document may conceptually look like:

```
{
    "_id": some_unique_id,
    "name": "Monitor",
    "price": 9000,
    "stock": 6
}
```

Searching for:

```
{"name": "Monitor"}
```

should return the matching document.

Your Python output may therefore look conceptually similar to:

```
{
    '_id': ...,
    'name': 'Monitor',
    'price': 9000,
    'stock': 6
}
```

The exact `_id` value may be different each time.

15. Hint Only

Think of MongoDB using this hierarchy:

MongoDB
↓
Database
↓
Collection
↓
Document

Your product document can start as an ordinary Python dictionary:

```
product = {  
    "name": ...,  
    "price": ...,  
    "stock": ...  
}
```

Then think about two operations:

```
How do I store one document?  
→ insert_one(...)  
  
How do I retrieve one matching document?  
→ find_one(...)
```

For your search condition, use a small dictionary such as:

```
{"name": "..."} 
```

The key idea for **Day 87** is:

MongoDB stores dictionary-like documents inside collections. Python dictionaries map naturally to this style of data, and basic operations such as `insert_one()` and `find_one()` allow you to store and retrieve documents.